ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 30

Vistas con Blade

- Qué es Blade
- Mostrar datos
- Condiciones
- Bucles y la variable $loop
- Layouts: @extends, @section y @yield
- Vistas parciales con @include
- Componentes
- Pasar datos a las vistas
- Recursos estáticos y Vite



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En la clase 29 dejamos las vistas con un problema evidente: todas repetían la cabecera, el menú y la hoja de estilos. Si mañana añadimos un enlace al menú, hay que tocar cinco ficheros. En nuestro MVC lo arreglamos con un *layout* y vistas parciales (clase 25). Hoy lo hacemos con **Blade**, el motor de plantillas de Laravel.

Blade no es un lenguaje nuevo: es HTML con unas cuantas **directivas** (`@if` , `@foreach` ...) que Laravel traduce a PHP normal y guarda en caché. Lo que aprendáis hoy lo vais a usar en todas las clases que quedan.

•

👌 Cómo vamos a trabajar



1

📘 Qué es Blade



2

👍 Mostrar datos



3

🤔 Condiciones



4

🚀 Bucles y la variable $loop



5

😮 Layouts: @extends, @section y @yield



6

📘 Vistas parciales con @include



7

🚀 Componentes



8

🔍 Pasar datos a las vistas



9

⚠️ Recursos estáticos y Vite



10

🚀 Ejercicios



11

👋 Lo que tenéis que llevaros de hoy





CLASE 30

👌 Cómo vamos a trabajar

Copiad el contenido de `ejemplos/` encima del proyecto `academia-laravel`:

Esquema

```
resources/views/layouts/app.blade.php          ← NUEVO: el layout de toda la web
resources/views/partials/menu.blade.php        ← NUEVO: el menú
resources/views/components/alerta.blade.php    ← NUEVO: componente <x-alerta>
resources/views/components/tarjeta-curso.blade.php ← NUEVO: componente <x-tarjeta-curso>
resources/views/ejemplos/directivas.blade.php  ← NUEVO: la página /blade para practicar
resources/views/paginas/ y cursos/             ← reescritas con el layout
config/academia.php                            ← añade los nombres de las categorías
app/Http/Controllers/PaginaController.php      ← la portada recibe los cursos destacados
routes/web.php                                 ← nueva ruta /blade
public/css/academia.css                        ← marca el enlace activo del menú
```



CLASE 30 · PUNTO 1

📘 Qué es Blade

Las vistas de Blade son ficheros que terminan en `.blade.php` y viven en `resources/views/` [1]. Cuando una vista se usa por primera vez, Laravel la **compila** a PHP normal y guarda el resultado en `storage/framework/views/`. Las siguientes veces usa directamente esa versión, así que Blade no hace más lenta la web.

En nuestro MVC (PHP puro)	En Blade
`<?= e($nombre) ?>`	`{{ $nombre }}`
`<?php if ($nota ⩾ 5): ?> ... <?php endif; ?>`	`@if ($nota ⩾ 5) ... @endif`
`<?php foreach ($cursos as $curso): ?> ... <?php endforeach; ?>`	`@foreach ($cursos as $curso) ... @endforeach`
`$this→parcial('menu')`	`@include('partials.menu')`
El layout que envuelve la vista	`@extends('layouts.app')`

Para practicar las directivas he preparado una página con todo junto: abrid `/blade` y tened al lado su vista, `ejemplos/directivas.blade.php`.



💡 Instalad en VS Code la extensión **Laravel Blade Snippets** o la oficial **Laravel**: colorean la sintaxis de Blade y autocompletan las directivas.



CLASE 30 · PUNTO 2

👍 Mostrar datos

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
1 <p>Escapado: {{ $html }}</p>
2 <p>Sin escapar: {!! $html !!}</p>
3 <p>Con valor por defecto: {{ $apodo ?? 'sin apodo' }}</p>
```

Si `$html` vale `'<strong>negrita</strong>'`, el resultado es:

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
1 <p>Escapado: &lt;strong&gt;negrita&lt;/strong&gt;</p>
2 <p>Sin escapar: <strong>negrita</strong></p>
3 <p>Con valor por defecto: sin apodo</p>
```

- `{{ }}` pasa el valor por `htmlspecialchars()`. Es lo que usaréis el 99 % de las veces: **protege contra XSS** (clase 13).
- `{!! !!}` imprime tal cual. Solo para HTML que hayamos generado **nosotros** y del que nos fiemos. **Nunca** con datos que haya escrito un usuario.
- Dentro de `{{ }}` cabe cualquier expresión de PHP: `{{ strtoupper($nombre) }}`, `{{ count($cursos) }}`, `{{ $precio * 1.21 }}`…

Otras tres cosas útiles:

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
{{-- Comentario de Blade: no llega al HTML, a diferencia de <!-- --> --}}

@php
    $media = array_sum($notas) / count($notas);
@endphp

<script>
    const alumnos = @json($alumnos);   // convierte el array de PHP en JSON para JavaScript
</script>
```



⚠️ `@php` existe, pero si os veis escribiendo mucho PHP en una vista, ese código debería estar en el controlador. La vista **muestra**, no calcula.



CLASE 30 · PUNTO 3

🤔 Condiciones

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```
@if ($nota ≥ 9)
    <p>Sobresaliente</p>
@elseif ($nota ≥ 5)
    <p>Aprobado</p>
@else
    <p>Suspenso</p>
@endif

@unless ($esAdmin)
    <p>No eres administrador.</p>     {{— @unless = "si NO" —}}
@endunless

@isset($apodo)
    <p>Tienes apodo.</p>              {{— existe y no es null —}}
@endisset

@empty($alumnos)
    <p>No hay alumnos.</p>            {{— está vacío —}}
@endempty

@switch($dia)
    @case('sábado')
    @case('domingo')
        <p>¡Fin de semana!</p>
        @break
    @default
        <p>Día de clase.</p>
@endswitch
```

Y una directiva muy práctica para las clases de CSS: `@class` añade solo las clases cuya condición sea verdadera. Así marcamos el enlace activo del menú:

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
<a href="{{ route('cursos.index') }}" @class(['activo' ⇒ request()
→routeIs('cursos.*')])>Cursos</a>
```

`request()→routeIs('cursos.*')` es verdadero si la ruta actual tiene un nombre que empieza por `cursos.`: otra ventaja de nombrar las rutas.



CLASE 30 · PUNTO 4

🚀 Bucles y la variable $loop

Tenemos los bucles de siempre (`@foreach` , `@for` , `@while`) y uno exclusivo de Blade, `@forelse` , que incluye el caso de que la lista esté vacía. Mirad el listado de cursos (`cursos/index.blade.php`):

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```blade
<div class="tarjetas">
    @forelse ($cursos as $curso)
        <x-tarjeta-curso :curso="$curso">
            @if ($loop->first)
                <p>⭐ El primero de la lista</p>
            @endif
        </x-tarjeta-curso>
    @empty
        <x-alerta tipo="error">No hay ningún curso que coincida con la búsqueda.</x-alerta>
    @endforelse
</div>
```

Dentro de cualquier `@foreach` o `@forelse` tenemos gratis la variable `$loop` , que nos ahorra contadores:

Propiedad	Qué vale
`$loop->iteration`	La vuelta actual, empezando en 1
`$loop->index`	La vuelta actual, empezando en 0
`$loop->count`	El total de elementos
`$loop->first` / `$loop->last`	Si es la primera / la última vuelta
`$loop->even` / `$loop->odd`	Si es una vuelta par / impar
`$loop->remaining`	Cuántas vueltas quedan

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```blade
@foreach ($alumnos as $alumno)
    <tr @class(['par' => $loop->even])>
        <td>{{ $loop->iteration }} de {{ $loop->count }}</td>
        <td>{{ $alumno }}</td>
    </tr>
@endforeach
```



CLASE 30 · PUNTO 5

😮 Layouts: @extends, @section y @yield

El **layout** es el esqueleto común de todas las páginas: cabecera, menú, pie y hoja de estilos. Lo guardamos en `layouts/app.blade.php` y marcamos con `@yield` los huecos que rellenará cada página [2]:

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('titulo', 'Inicio') · {{ config('app.name') }}</title>
    <link rel="stylesheet" href="{{ asset('css/academia.css') }}">
    @stack('estilos')
</head>
<body>
    @include('partials.menu')

    <main>
        @yield('contenido')
    </main>

    <footer>
        © {{ date('Y') }} {{ config('app.name') }} · Santander
        · <a href="{{ route('aviso-legal') }}">Aviso legal</a>
    </footer>

    @stack('scripts')
</body>
</html>
```

Cada página dice qué layout usa con `@extends` y rellena los huecos con `@section` (`contacto.blade.php`):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```
@extends('layouts.app')

@section('titulo', 'Contacto')

@section('contenido')
    <h1>📞 Contacto</h1>
    <p>Teléfono: {{ $telefono }}</p>
    <p>Correo: <a href="mailto:{{ $email }}">{{ $email }}</a></p>
@endsection
```

- `@section('titulo', 'Contacto')` es la forma corta, para un texto.
- `@section('contenido') ... @endsection` es la forma larga, para un bloque de HTML.
- `@yield('titulo', 'Inicio')` usa `'Inicio'` si la página no define esa sección.

Pilas: @stack y @push

¿Y si una sola página necesita un poco de CSS o de JavaScript propio? La empuja a una **pila** con `@push`, y el layout la pinta donde esté el `@stack`:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```
@push('estilos')
    <style>
        .par { background: #fff4e6; }
    </style>
@endpush
```

Ahora el menú está en **un solo sitio**. ¡Añadid un enlace y comprobad que aparece en todas las páginas! 🎉



Un trozo de vista que se repite se guarda aparte y se incluye con `@include`. Nuestro menú está en `partials/menu.blade.php`:

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
<header>
    <a class="marca" href="{{ route('inicio') }}">🎓 {{ config('app.name') }}</a>
    <nav>
        <a href="{{ route('cursos.index') }}" @class(['activo' ⇒ request()
->routeIs('cursos.*')])>Cursos</a>
        <a href="{{ route('contacto') }}" @class(['activo' ⇒ request()
->routeIs('contacto')])>Contacto</a>
    </nav>
</header>
```

La vista incluida ve las mismas variables que la que la incluye, y además se le pueden pasar más: `@include('partials.aviso', ['texto' ⇒ 'Plazas limitadas'])`.



CLASE 30 · PUNTO 7

🚀 Componentes

Los **componentes** son la evolución de los parciales: piezas reutilizables que se usan **como si fueran etiquetas HTML** y que reciben datos a través de atributos [3]. Los más sencillos (componentes **anónimos**) son una vista dentro de `resources/views/components/`.

Un componente con contenido:

`components/alerta.blade.php`:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```
@props(['tipo' => 'info'])

<div {{ $attributes->merge(['class' => "alerta alerta-$tipo"]) }}>
    {{ $slot }}
</div>
```

Y se usa así:

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
<x-alerta tipo="exito">
    ¡Matrícula abierta! Tenemos <strong>{{ $totalCursos }}</strong> cursos esperándoos.
</x-alerta>
```

- El fichero `components/alerta.blade.php` se convierte en la etiqueta `<x-alerta>`.
- `@props` declara los atributos que acepta, con sus valores por defecto.
- `$slot` es lo que escribimos entre la etiqueta de apertura y la de cierre.
- `$attributes` son el resto de atributos (una `id`, otra clase...): `merge()` los junta con los nuestros.

Un componente que recibe datos:

`components/tarjeta-curso.blade.php`:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```blade
@props(['curso'])

<article class="tarjeta">
    <h3>{{ $curso['titulo'] }}</h3>
    <p>
        <span class="etiqueta">{{ config("academia.categorias.{$curso['categoria']}") }}
</span>
        {{ $curso['horas'] }} horas
    </p>
    <p><strong>{{ number_format($curso['precio'], 2, ',', '.') }} €</strong></p>

    {{-- Si quien usa el componente le pasa contenido, se pinta aquí --}}
    {{ $slot }}

    <a class="boton" href="{{ route('cursos.show', $curso['id']) }}">Ver curso</a>
</article>
```

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```blade
<x-tarjeta-curso :curso="$curso" />
```



⚠️ **Fijaos en los dos puntos.** `curso="$curso"` le pasaría el **texto** «$curso». Con `:curso="$curso"`, lo de dentro de las comillas se evalúa como **PHP** y se pasa la variable. Lo mismo con `:href="https://github.com/ramonabramo/cursoPoo2026/blob/main/clase-30-lara-vel-blade/route('inicio')"`.

La tarjeta aparece en la portada (los tres cursos destacados) y en el listado. Si mañana cambiamos su diseño, cambia en los dos sitios.

	`@include`	Componente
Uso	`@include('partials.menu')`	`<x-alerta tipo="error">...</x-alerta>`
Recibe datos	Las variables de la vista + un array	Atributos declarados con `@props`
Contenido dentro	No	Sí, con `$slot`
Para...	Trozos fijos (menú, pie)	Piezas reutilizables con variaciones



💡 También hay componentes **con clase**, que se crean con `php artisan make:component` y permiten lógica en PHP. Para la mayoría de los casos, los anónimos bastan.



CLASE 30 · PUNTO 8

🔍 Pasar datos a las vistas

Ya lo hemos hecho muchas veces con un array, que es la forma más clara:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
return view('paginas.contacto', [
    'telefono' => config('academia.telefono'),
    'email' => config('academia.email'),
]);
```

Pero os encontraréis estas otras en ejemplos de internet:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
return view('cursos.show')->with('curso', $curso);    // dato a dato
return view('cursos.index', compact('cursos', 'buscar'));  // compact() crea el array con esos nombres
```

Y para datos que necesitan **todas** las vistas (por ejemplo, el teléfono en el pie), se puede usar `View::share('telefono', ...)` en el método `boot()` de `AppServiceProvider`. Lo veremos en la clase 35.

La portada ahora recibe también los cursos destacados (`PaginaController.php`):

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```php
public function inicio(): View
{
    $cursos = config('academia.cursos');

    return view('paginas.inicio', [
        'totalCursos' => count($cursos),
        'destacados' => array_slice($cursos, 0, 3),
    ]);
}
```



CLASE 30 · PUNTO 9

⚠️ Recursos estáticos y Vite

Todo lo que está en `public/` se sirve directamente. Para enlazarlo, **siempre con** `asset()`, que genera la dirección completa y funciona aunque la web esté en una subcarpeta:

plantilla.php — php-paso-a-paso/clase30 — Visual Studio Code

plantilla.php

```
<link rel="stylesheet" href="{{ asset('css/academia.css') }}">
<img src="{{ asset('img/logo.png') }}" alt="Logo de la academia">
```

Si un día pasáis a **Vite** (clase 28), el cambio en el layout es una línea:

ejemplo.php — php-paso-a-paso/clase30 — Visual Studio Code

ejemplo.php

```
{{-- Antes --}}
<link rel="stylesheet" href="{{ asset('css/academia.css') }}">

{{-- Con Vite: npm install y npm run dev --}}
@vite(['resources/css/app.css', 'resources/js/app.js'])
```

Con `npm run dev`, Vite vigila `resources/css` y `resources/js` y recarga el navegador cada vez que guardáis. Con `npm run build`, genera los ficheros finales en `public/build/` y `@vite` los enlaza solo [4].



CLASE 30 · PUNTO 10

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Un componente botón.** Cread el componente `<x-boton>`, que reciba `href` y un `tipo` (`principal`, `secundario` o `peligro`) y pinte un enlace con la clase CSS adecuada. Usadlo en la ficha del curso para «Volver a los cursos» y «Pedir información». → `components/boton.blade.php` y `cursos/show.blade.php`
- ☐ **02 · Tabla de precios.** Cread la página `/precios` con una tabla de todos los cursos: número de fila (`$loop`), título con enlace, horas, precio y precio con IVA, con las filas pares de otro color (`@class` y `@push`) y una fila final con los totales. → `paginas/precios.blade.php` y `routes/web.php`
- ☐ **03 · Nuestra propia página 404.** Si existe la vista `resources/views/errors/404.blade.php`, Laravel la usa para los errores 404. Cread una que use el layout y una alerta, y probadla con `/cursos/99`. → `errors/404.blade.php`



🏆 **Reto:** cread un componente `<x-tarjeta>` genérico con un **slot con nombre** para el título (`<x-slot:titulo>...</x-slot>`) y el contenido en `$slot`, y usadlo en la página de contacto. Buscad «Named Slots» en la documentación de Blade.



CLASE 30 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Blade	HTML con directivas; se compila a PHP y se guarda en caché
`{{ }}` / `{!! !!}`	Escapado (siempre) / sin escapar (solo HTML nuestro)
`@if`, `@unless`, `@isset`, `@switch`	Condiciones sin abrir y cerrar `<?php ?>`
`@forelse ... @empty`	Un bucle con el caso «no hay nada» incluido
`$loop`	Vuelta actual, total, primera, última, par... gratis en cada bucle
`@extends` + `@section` + `@yield`	Un layout para todas las páginas
`@stack` + `@push`	CSS o JS propio de una página
`@include`	Trozos fijos, como el menú
`<x-componente>`	Piezas reutilizables con `@props` y `$slot`
`:atributo="$variable"`	Los dos puntos pasan PHP, no texto
`asset()` / `@vite`	Enlazar ficheros de `public/` / compilados por Vite

📚 Fuentes y documentación oficial

1. Laravel: Blade Templates
2. Laravel: Blade, Layouts Using Template Inheritance
3. Laravel: Blade, Components
4. Laravel: Asset Bundling (Vite)
5. Laravel: Views